Lab3 linear regression

Jupyter Notebook — original cells shown separately and in order

CELL 01 | CODE | EXECUTION COUNT: 77

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LinearRegression
from sklearn.metrics import accuracy_score, classification_report
from scipy import stats
from sklearn.metrics import mean_squared_error, r2_score

import warnings
warnings.filterwarnings('ignore')
```

CELL 02 | CODE | EXECUTION COUNT: 78

```
df=pd.read_csv("BostonHousing 3.csv")
```

CELL 03 | CODE | EXECUTION COUNT: 79

```
df
```

OUTPUT

```
        crim    zn  indus  chas    nox     rm   age     dis  rad  tax  \
0    0.00632  18.0   2.31     0  0.538  6.575  65.2  4.0900    1  296
1    0.02731   0.0   7.07     0  0.469  6.421  78.9  4.9671    2  242
2    0.02729   0.0   7.07     0  0.469  7.185  61.1  4.9671    2  242
3    0.03237   0.0   2.18     0  0.458  6.998  45.8  6.0622    3  222
4    0.06905   0.0   2.18     0  0.458  7.147  54.2  6.0622    3  222
..       ...   ...    ...   ...    ...    ...   ...     ...  ...  ...
501  0.06263   0.0  11.93     0  0.573  6.593  69.1  2.4786    1  273
502  0.04527   0.0  11.93     0  0.573  6.120  76.7  2.2875    1  273
503  0.06076   0.0  11.93     0  0.573  6.976  91.0  2.1675    1  273
504  0.10959   0.0  11.93     0  0.573  6.794  89.3  2.3889    1  273
505  0.04741   0.0  11.93     0  0.573  6.030  80.8  2.5050    1  273

     ptratio       b  lstat  medv
0       15.3  396.90   4.98  24.0
1       17.8  396.90   9.14  21.6
2       17.8  392.83   4.03  34.7
3       18.7  394.63   2.94  33.4
4       18.7  396.90   5.33  36.2
..       ...     ...    ...   ...
501     21.0  391.99   9.67  22.4
502     21.0  396.90   9.08  20.6
503     21.0  396.90   5.64  23.9
504     21.0  393.45   6.48  22.0
505     21.0  396.90   7.88  11.9

[506 rows x 14 columns]
```

CELL 04 | CODE | EXECUTION COUNT: 80

```
df.head()
```

OUTPUT

```
      crim    zn  indus  chas    nox     rm   age     dis  rad  tax  ptratio  \
0  0.00632  18.0   2.31     0  0.538  6.575  65.2  4.0900    1  296     15.3
1  0.02731   0.0   7.07     0  0.469  6.421  78.9  4.9671    2  242     17.8
2  0.02729   0.0   7.07     0  0.469  7.185  61.1  4.9671    2  242     17.8
3  0.03237   0.0   2.18     0  0.458  6.998  45.8  6.0622    3  222     18.7
4  0.06905   0.0   2.18     0  0.458  7.147  54.2  6.0622    3  222     18.7

        b  lstat  medv
0  396.90   4.98  24.0
1  396.90   9.14  21.6
2  392.83   4.03  34.7
3  394.63   2.94  33.4
4  396.90   5.33  36.2
```

CELL 05 | CODE | EXECUTION COUNT: 81

```
df.tail()
```

OUTPUT

```
        crim    zn  indus  chas    nox     rm   age     dis  rad  tax  ptratio  \
501  0.06263  0.0  11.93     0  0.573  6.593  69.1  2.4786    1  273     21.0
502  0.04527  0.0  11.93     0  0.573  6.120  76.7  2.2875    1  273     21.0
503  0.06076  0.0  11.93     0  0.573  6.976  91.0  2.1675    1  273     21.0
504  0.10959  0.0  11.93     0  0.573  6.794  89.3  2.3889    1  273     21.0
505  0.04741  0.0  11.93     0  0.573  6.030  80.8  2.5050    1  273     21.0

          b  lstat  medv
501  391.99   9.67  22.4
502  396.90   9.08  20.6
503  396.90   5.64  23.9
504  393.45   6.48  22.0
505  396.90   7.88  11.9
```

CELL 06 | CODE | EXECUTION COUNT: 82

```
df.describe
```

OUTPUT

```
<bound method NDFrame.describe of         crim    zn  indus  chas    nox     rm   age     dis  rad  tax  \
0    0.00632  18.0   2.31     0  0.538  6.575  65.2  4.0900    1  296
1    0.02731   0.0   7.07     0  0.469  6.421  78.9  4.9671    2  242
2    0.02729   0.0   7.07     0  0.469  7.185  61.1  4.9671    2  242
3    0.03237   0.0   2.18     0  0.458  6.998  45.8  6.0622    3  222
4    0.06905   0.0   2.18     0  0.458  7.147  54.2  6.0622    3  222
..       ...   ...    ...   ...    ...    ...   ...     ...  ...  ...
501  0.06263   0.0  11.93     0  0.573  6.593  69.1  2.4786    1  273
502  0.04527   0.0  11.93     0  0.573  6.120  76.7  2.2875    1  273
503  0.06076   0.0  11.93     0  0.573  6.976  91.0  2.1675    1  273
504  0.10959   0.0  11.93     0  0.573  6.794  89.3  2.3889    1  273
505  0.04741   0.0  11.93     0  0.573  6.030  80.8  2.5050    1  273

     ptratio       b  lstat  medv
0       15.3  396.90   4.98  24.0
1       17.8  396.90   9.14  21.6
2       17.8  392.83   4.03  34.7
3       18.7  394.63   2.94  33.4
4       18.7  396.90   5.33  36.2
..       ...     ...    ...   ...
501     21.0  391.99   9.67  22.4
502     21.0  396.90   9.08  20.6
503     21.0  396.90   5.64  23.9
504     21.0  393.45   6.48  22.0
505     21.0  396.90   7.88  11.9

[506 rows x 14 columns]>
```

CELL 07 | CODE | EXECUTION COUNT: 83

```
df.size
```

OUTPUT

```
7084
```

CELL 08 | CODE | EXECUTION COUNT: 84

```
df.dtypes
```

OUTPUT

```
crim       float64
zn         float64
indus      float64
chas         int64
nox        float64
rm         float64
age        float64
dis        float64
rad          int64
tax          int64
ptratio    float64
b          float64
lstat      float64
medv       float64
dtype: object
```

CELL 09 | CODE | EXECUTION COUNT: 85

```
df.info()
```

OUTPUT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   crim     506 non-null    float64
 1   zn       506 non-null    float64
 2   indus    506 non-null    float64
 3   chas     506 non-null    int64  
 4   nox      506 non-null    float64
 5   rm       501 non-null    float64
 6   age      506 non-null    float64
 7   dis      506 non-null    float64
 8   rad      506 non-null    int64  
 9   tax      506 non-null    int64  
 10  ptratio  506 non-null    float64
 11  b        506 non-null    float64
 12  lstat    506 non-null    float64
 13  medv     506 non-null    float64
dtypes: float64(11), int64(3)
memory usage: 55.5 KB
```

CELL 10 | CODE | EXECUTION COUNT: 86

```
df.shape
```

OUTPUT

```
(506, 14)
```

CELL 11 | CODE | EXECUTION COUNT: 87

```
df.notnull()
```

OUTPUT

```
      crim    zn  indus  chas   nox    rm   age   dis   rad   tax  ptratio  \
0     True  True   True  True  True  True  True  True  True  True     True   
1     True  True   True  True  True  True  True  True  True  True     True   
2     True  True   True  True  True  True  True  True  True  True     True   
3     True  True   True  True  True  True  True  True  True  True     True   
4     True  True   True  True  True  True  True  True  True  True     True   
..     ...   ...    ...   ...   ...   ...   ...   ...   ...   ...      ...   
501   True  True   True  True  True  True  True  True  True  True     True   
502   True  True   True  True  True  True  True  True  True  True     True   
503   True  True   True  True  True  True  True  True  True  True     True   
504   True  True   True  True  True  True  True  True  True  True     True   
505   True  True   True  True  True  True  True  True  True  True     True   

         b  lstat  medv  
0     True   True  True  
1     True   True  True  
2     True   True  True  
3     True   True  True  
4     True   True  True  
..     ...    ...   ...  
501   True   True  True  
502   True   True  True  
503   True   True  True  
504   True   True  True  
505   True   True  True  

[506 rows x 14 columns]
```

CELL 12 | CODE | EXECUTION COUNT: 88

```
df.isnull().sum()
```

OUTPUT

```
crim       0
zn         0
indus      0
chas       0
nox        0
rm         5
age        0
dis        0
rad        0
tax        0
ptratio    0
b          0
lstat      0
medv       0
dtype: int64
```

CELL 13 | MARKDOWN

(Empty cell)

CELL 14 | CODE | EXECUTION COUNT: 89

```
df.columns
```

OUTPUT

```
Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis', 'rad', 'tax',
       'ptratio', 'b', 'lstat', 'medv'],
      dtype='object')
```

CELL 15 | CODE | EXECUTION COUNT: 90

```
df.fillna(df.mean(numeric_only=True),inplace=True)
df
```

OUTPUT

```
        crim    zn  indus  chas    nox     rm   age     dis  rad  tax  \
0    0.00632  18.0   2.31     0  0.538  6.575  65.2  4.0900    1  296
1    0.02731   0.0   7.07     0  0.469  6.421  78.9  4.9671    2  242
2    0.02729   0.0   7.07     0  0.469  7.185  61.1  4.9671    2  242
3    0.03237   0.0   2.18     0  0.458  6.998  45.8  6.0622    3  222
4    0.06905   0.0   2.18     0  0.458  7.147  54.2  6.0622    3  222
..       ...   ...    ...   ...    ...    ...   ...     ...  ...  ...
501  0.06263   0.0  11.93     0  0.573  6.593  69.1  2.4786    1  273
502  0.04527   0.0  11.93     0  0.573  6.120  76.7  2.2875    1  273
503  0.06076   0.0  11.93     0  0.573  6.976  91.0  2.1675    1  273
504  0.10959   0.0  11.93     0  0.573  6.794  89.3  2.3889    1  273
505  0.04741   0.0  11.93     0  0.573  6.030  80.8  2.5050    1  273

     ptratio       b  lstat  medv
0       15.3  396.90   4.98  24.0
1       17.8  396.90   9.14  21.6
2       17.8  392.83   4.03  34.7
3       18.7  394.63   2.94  33.4
4       18.7  396.90   5.33  36.2
..       ...     ...    ...   ...
501     21.0  391.99   9.67  22.4
502     21.0  396.90   9.08  20.6
503     21.0  396.90   5.64  23.9
504     21.0  393.45   6.48  22.0
505     21.0  396.90   7.88  11.9

[506 rows x 14 columns]
```

CELL 16 | CODE | EXECUTION COUNT: 91

```
df.isnull().sum()
```

OUTPUT

```
crim       0
zn         0
indus      0
chas       0
nox        0
rm         0
age        0
dis        0
rad        0
tax        0
ptratio    0
b          0
lstat      0
medv       0
dtype: int64
```

CELL 17 | CODE | EXECUTION COUNT: 92

```
rm_z_scores = np.abs(stats.zscore(df['rm']))
outliers_z = df[rm_z_scores > 3]
print(f"Z-Score Method found {len(outliers_z)} outliers in the 'rm' column.")
```

OUTPUT

```
Z-Score Method found 8 outliers in the 'rm' column.
```

CELL 18 | CODE | EXECUTION COUNT: 93

```
sns.boxplot(x=df['medv'])
plt.title("Outliers in Boston Housing Prices (medv)")
plt.xlabel("Median Home Value ($1000s)")
plt.show()
```

OUTPUT

```
<Figure size 640x480 with 1 Axes>
```

CELL 19 | CODE | EXECUTION COUNT: 94

```
co=df.corr()
print(co)
```

OUTPUT

```
             crim        zn     indus      chas       nox        rm       age  \
crim     1.000000 -0.200469  0.406583 -0.055892  0.420972 -0.219261  0.352734
zn      -0.200469  1.000000 -0.533828 -0.042697 -0.516604  0.310847 -0.569537
indus    0.406583 -0.533828  1.000000  0.062938  0.763651 -0.392052  0.644779
chas    -0.055892 -0.042697  0.062938  1.000000  0.091203  0.091434  0.086518
nox      0.420972 -0.516604  0.763651  0.091203  1.000000 -0.302051  0.731470
rm      -0.219261  0.310847 -0.392052  0.091434 -0.302051  1.000000 -0.239632
age      0.352734 -0.569537  0.644779  0.086518  0.731470 -0.239632  1.000000
dis     -0.379670  0.664408 -0.708027 -0.099176 -0.769230  0.202268 -0.747881
rad      0.625505 -0.311948  0.595129 -0.007368  0.611441 -0.210353  0.456022
tax      0.582764 -0.314563  0.720760 -0.035587  0.668023 -0.292178  0.506456
ptratio  0.289946 -0.391679  0.383248 -0.121515  0.188933 -0.356065  0.261515
b       -0.385064  0.175520 -0.356977  0.048788 -0.380051  0.127998 -0.273534
lstat    0.455621 -0.412995  0.603800 -0.053929  0.590879 -0.614526  0.602339
medv    -0.388305  0.360445 -0.483725  0.175260 -0.427321  0.695375 -0.376955

               dis       rad       tax   ptratio         b     lstat      medv
crim    -0.379670  0.625505  0.582764  0.289946 -0.385064  0.455621 -0.388305
zn       0.664408 -0.311948 -0.314563 -0.391679  0.175520 -0.412995  0.360445
indus   -0.708027  0.595129  0.720760  0.383248 -0.356977  0.603800 -0.483725
chas    -0.099176 -0.007368 -0.035587 -0.121515  0.048788 -0.053929  0.175260
nox     -0.769230  0.611441  0.668023  0.188933 -0.380051  0.590879 -0.427321
rm       0.202268 -0.210353 -0.292178 -0.356065  0.127998 -0.614526  0.695375
age     -0.747881  0.456022  0.506456  0.261515 -0.273534  0.602339 -0.376955
dis      1.000000 -0.494588 -0.534432 -0.232471  0.291512 -0.496996  0.249929
rad     -0.494588  1.000000  0.910228  0.464741 -0.444413  0.488676 -0.381626
tax     -0.534432  0.910228  1.000000  0.460853 -0.441808  0.543993 -0.468536
ptratio -0.232471  0.464741  0.460853  1.000000 -0.177383  0.374044 -0.507787
b        0.291512 -0.444413 -0.441808 -0.177383  1.000000 -0.366087  0.333461
lstat   -0.496996  0.488676  0.543993  0.374044 -0.366087  1.000000 -0.737663
medv     0.249929 -0.381626 -0.468536 -0.507787  0.333461 -0.737663  1.000000
```

CELL 20 | CODE | EXECUTION COUNT: 95

```
plt.figure(figsize=(12,8))
sns.heatmap(co,annot=True,cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

OUTPUT

```
<Figure size 1200x800 with 2 Axes>
```

CELL 21 | CODE | EXECUTION COUNT: 96

```
plt.figure(figsize=(8,6))
sns.histplot(df["medv"], bins=30, kde=True)
plt.title("Distribution of House Prices")
plt.xlabel("House Price")
plt.ylabel("Frequency")
plt.show()
```

OUTPUT

```
<Figure size 800x600 with 1 Axes>
```

CELL 22 | CODE | EXECUTION COUNT: 97

```
X = df.drop("medv",axis=1)
y = df["medv"]
```

CELL 23 | CODE | EXECUTION COUNT: 98

```
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.20,random_state=42)
print("Training Data Shape :", X_train.shape)
print("Testing Data Shape  :", X_test.shape)
```

OUTPUT

```
Training Data Shape : (404, 13)
Testing Data Shape  : (102, 13)
```

CELL 24 | CODE | EXECUTION COUNT: 99

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

CELL 25 | CODE | EXECUTION COUNT: 100

```
model = LinearRegression()
model.fit(X_train, y_train)
print("Model Training Completed Successfully.")
```

OUTPUT

```
Model Training Completed Successfully.
```

CELL 26 | CODE | EXECUTION COUNT: 101

```
print("Intecept:")
print(model.intercept_)
coefficients = pd.DataFrame({"Feature": X.columns,"Coefficient": model.coef_})
coefficients
```

OUTPUT

```
Intecept:
30.29636237179123
```

	Feature	Coefficient
0	crim	-0.113051
1	zn	0.030209
2	indus	0.041726
3	chas	2.787586
4	nox	-17.188929
5	rm	4.429193
6	age	-0.006109
7	dis	-1.442141
8	rad	0.262753
9	tax	-0.010647
10	ptratio	-0.918081
11	b	0.012349
12	lstat	-0.508973

CELL 27 | CODE | EXECUTION COUNT: 102

```
y_pred=model.predict(X_test)
print("First 10 Predicted Values")
print(y_pred[:10])
```

OUTPUT

```
First 10 Predicted Values
[28.98141872 36.01423173 14.82430919 25.0229357  18.77077366 23.25088901
 17.65921355 14.3312157  23.01799598 20.61923725]
```

CELL 28 | CODE | EXECUTION COUNT: 103

```
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error :", mse)
print("Root Mean Squared Error :", rmse)
print("R2 Score :", r2)
```

OUTPUT

```
Mean Squared Error : 24.404825188146287
Root Mean Squared Error : 4.940124005340988
R2 Score : 0.6672089705941906
```

CELL 29 | CODE | EXECUTION COUNT: 104

```
comparison = pd.DataFrame({"Actual": y_test.values,"Predicted": y_pred})
comparison.head(15)
```

OUTPUT

	Actual	Predicted
0	23.6	28.981419
1	32.4	36.014232
2	13.6	14.824309
3	22.8	25.022936
4	16.1	18.770774
5	20.0	23.250889
6	17.8	17.659214
7	14.0	14.331216
8	19.6	23.017996
9	16.8	20.619237
10	21.5	24.914775
11	18.9	18.657450
12	7.0	-6.080249
13	21.2	21.756694
14	18.5	19.236084

CELL 30 | CODE | EXECUTION COUNT: 105

```
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         color='red',
         linewidth=2)
plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title("Actual vs Predicted House Price")
plt.show()
```

OUTPUT

```
<Figure size 800x600 with 1 Axes>
```

CELL 31 | CODE | EXECUTION COUNT: 106

```
residuals = y_test - y_pred
plt.figure(figsize=(8,6))
plt.scatter(y_pred, residuals)
plt.axhline(y=0, color='red')
plt.xlabel("Predicted Values")
plt.ylabel("Residual Error")
plt.title("Residual Plot")
plt.show()
```

OUTPUT

```
<Figure size 800x600 with 1 Axes>
```

CELL 32 | CODE | EXECUTION COUNT: 107

```
print("="*50)
print("Multiple Linear Regression performance")
print("="*50)
print("Mean Squared Error(mse):",round(mse,4))
print ("Root mean squared error(RMSE):",round(rmse,4))
print("R^2 Score:",round(r2,4))
```

OUTPUT

```
==================================================
Multiple Linear Regression performance
==================================================
Mean Squared Error(mse): 24.4048
Root mean squared error(RMSE): 4.9401
R^2 Score: 0.6672
```

CELL 33 | CODE | EXECUTION COUNT: 108

```
print("Total Model Acurracy in Percentage:",r2*100)
```

OUTPUT

```
Total Model Acurracy in Percentage: 66.72089705941906
```